Consistency Models and the CAP Theorem: Quorum Tuning, Network Partitions, and PACELC Trade-offs in a Replicated Key-Value Store

Danil Shumayev
School of Digital Technologies, Cybersecurity, 4th Year
CSS4004 Distributed Systems
Ernazaruly Bakdaulet
November 11, 2026

Consistency Models and the CAP Theorem: Quorum Tuning, Network Partitions, and PACELC Trade-offs in a Replicated Key-Value Store

Design Summary

The key-value store extends the Assignment 2 replicas: every server.py process now also serves the KeyValueStore service from kvstore.proto. Node A (port 50051) holds the lease for the keyspace [a-z]; B and C are followers. Every value is stored as a version (value, timestamp, origin node), where the timestamp is a hybrid clock in microseconds. Two versions of a key are reconciled with Last-Write-Wins: the higher timestamp wins, and the node name breaks ties, so all replicas that see the same versions end in the same state.

Writes

A QUORUM_CP write is forwarded to the leaseholder, which sends it to both followers in parallel and commits it only when W nodes, itself included, hold it (W = 2 by default). Without a quorum the leaseholder does not apply the write and returns success = false with ReplicaUnavailableError. An ALWAYS_AP write is applied at once by the node that receives it and pushed to every peer it can reach; acks_received counts the nodes that hold it.

Reads

A LINEARIZABLE read is answered only by the leaseholder; a follower proxies it there and returns UNAVAILABLE if A cannot be reached. A FOLLOWER_STALE read is answered from the local copy. A follower measures its lag as the time since it last heard from the leaseholder; if the lag is above max_staleness_ms it returns FAILED_PRECONDITION with STALE_READ_EXCEEDED.

Anti-Entropy and Partition Simulation

Every 200 ms each node pushes its whole state to its peers, which merge it with Last-Write-Wins. This heals divergence after a partition and also serves as the leaseholder's heartbeat. In tests/test_cap.py the nodes are real processes that reach each other only through TCP proxies, one per directed link; cutting the links between DC-1 {A, B} and DC-2 {C} produces a network partition, as described in Task 3.

Section 1: Test Results and Quorum Verification

Table 1

Test Environment

Item	Value
Machine	Apple MacBook Air (Mac16,12), Apple M4, 10 CPU cores, 16 GB RAM
Operating system	macOS 27.3 (build 26C5035r), arm64
Python	3.14.7 (Homebrew), virtual environment .venv
Key packages	grpcio 1.84.0, grpcio-tools 1.84.0, protobuf 7.36.2, pytest 9.1.1 (Appendix A)
Cluster	3 server.py processes on localhost: A (leaseholder for [a-z]), B, C; N = 3

Table 2

Test Cases in tests/test_cap.py (18 of 18 Passed)

ID	Test	Scenario	Expected	Actual	Result
CAP-01	`test_cp_write_healthy_cluster_reaches_all_replicas`	No partition; QUORUM_CP Put sent to follower B	Success, 3 acks, value on every node	success, acks = 3 (B forwarded to A); A = B = C = v1	Pass
CAP-02	`test_cp_write_on_majority_partition_succeeds`	DC-1 {A,B} | DC-2 {C}; QUORUM_CP Put to A	Success, acks_received = 2	success, acks = 2; B = dc1, C has no value	Pass
CAP-03	`test_cp_write_on_minority_partition_fails`	Same partition; QUORUM_CP Put to C	Failure: ReplicaUnavailableError / quorum loss	success = false, `ReplicaUnavailableError: quorum lost (leaseholder A unreachable from C)`; value on no node	Pass
CAP-04	`test_ap_write_on_minority_partition_is_accepted`	Same partition; ALWAYS_AP Put to C	Success, acks_received = 1	success, acks = 1; C = dc2, A has no value (divergence)	Pass
CAP-05	`test_lww_merges_divergent_values_after_heal [dc2 newer]`	CP dc1 (ts 1000) on A, AP dc2 (ts 2000) on C, then heal	LWW: every node converges to dc2	A = B = C = dc2; linearizable read ts = 2000	Pass
CAP-06	`test_lww_merges_divergent_values_after_heal [dc1 newer]`	CP dc1 (ts 2000) on A, AP dc2 (ts 1000) on C, then heal	LWW: the minority write is discarded	A = B = C = dc1; linearizable read ts = 2000	Pass
CAP-07	`test_follower_read_within_staleness_bound_is_served_locally`	FOLLOWER_STALE Get on C, bound 5,000 ms	Served by C, is_stale = true	v1, is_stale = true; LINEARIZABLE via C: is_stale = false	Pass
CAP-08	`test_stale_read_exceeded_on_isolated_follower`	C partitioned for 0.5 s; FOLLOWER_STALE Get, bound 300 ms, then 5,000 ms	STALE_READ_EXCEEDED, then served	FAILED_PRECONDITION `STALE_READ_EXCEEDED`; bound 5,000 ms: v1	Pass

ID	Test	Scenario	Expected	Actual	Result
CAP-09	`test_linearizable_read_unavailable_on_minority`	Partition; LINEARIZABLE Get on C and on B	C unavailable, B answers	C: UNAVAILABLE; B: v1	Pass
CAP-10 to 18	`test_quorum_intersection [W, R] (9 cases)`	Every (W, R) for N = 3; every R-subset of nodes is read	R + W > N: never stale; R + W ≤ N: stale subset exists	As expected in all 9 cases (Table 3)	Pass

Note. Nodes are real server.py processes. Each node reaches its peers through a TCP proxy (one per directed link); a partition makes the proxies between DC-1 and DC-2 drop open connections and refuse new ones. tests/test_cap.py: 18 passed in 3.1 s. Full suite including the 17 Assignment 2 tests: 35 passed in 8.7 s, with no failures in 10 consecutive runs.

Table 3

Quorum Verification (N = 3, Measured)

W	R	R + W	R + W > N	Read quorums tested	Stale quorums	Which	Observed consistency	Result
1	1	2	No	3	2	{B}, {C}	Eventual (stale possible)	Pass
1	2	3	No	3	1	{B,C}	Eventual (stale possible)	Pass
1	3	4	Yes	1	0	none	Strong (always fresh)	Pass
2	1	3	No	3	1	{C}	Eventual (stale possible)	Pass
2	2	4	Yes	3	0	none	Strong (always fresh)	Pass
2	3	5	Yes	1	0	none	Strong (always fresh)	Pass
3	1	4	Yes	3	0	none	Strong (always fresh)	Pass
3	2	5	Yes	3	0	none	Strong (always fresh)	Pass
3	3	6	Yes	1	0	none	Strong (always fresh)	Pass

Note. Procedure: write 'old' to all three nodes; cut the network so that the QUORUM_CP write of 'new' reaches exactly W nodes (A; A and B; or all three); heal with anti-entropy switched off; then for every R-subset of {A, B, C} read the local copies and keep the highest timestamp. A quorum is stale when the result is 'old'.

All nine combinations behave as quorum arithmetic predicts. Whenever R + W > N, none of the tested read quorums returned the old value. Whenever R + W ≤ N, at least one read quorum consisted only of nodes that missed the write and returned the old value; for W = 2 and R = 1 this is the single quorum {C}, the node that was cut off during the write.

Section 2: Performance Evaluation

The benchmark runs a three-node cluster on localhost and sends 1,000 reads per scenario to follower B, first in a healthy cluster, then right after leaseholder A is killed, and again once A has been down for longer than the 5-second staleness bound.

Table 4

PACELC Verification: Read Latency and Availability (1,000 Reads per Cell)

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%): healthy	A crashed, < 5 s	A down, > 5 s
Linearizable	Leaseholder (Remote)	0.273	0.314	100.0	0.0	0.0
Follower Stale	Local Follower	0.126	0.143	100.0	100.0	0.0

Note. Script: perf_cap_bench.py. The client sends every read to follower B; a linearizable read is proxied by B to leaseholder A, a follower read is answered by B with max_staleness_ms = 5000. Latency is measured over successful reads in the healthy cluster, after 50 warm-up reads; p95 is the value at index ceil(0.95 × n). Failed linearizable reads returned UNAVAILABLE; failed follower reads in the last phase returned STALE_READ_EXCEEDED.

Else (no partition): latency versus consistency. A follower read was 2.2 times faster at the median (0.126 ms against 0.273 ms) because it skips the extra hop from B to the leaseholder. On localhost this hop costs only a fraction of a millisecond; between data centers it would cost a full wide-area round trip, so the gap would be much larger. The price of the faster read is that it may return a value up to max_staleness_ms old.

Partition or crash: availability versus consistency. After the leaseholder crashed, every linearizable read failed with UNAVAILABLE (0 % availability), while follower reads stayed at 100 %. The bound limits this: once B had not heard from the leaseholder for more than 5 seconds, follower reads were also refused with STALE_READ_EXCEEDED. The bound sets how much staleness the application accepts in return for availability.

Section 3: Conceptual Questions

1. Why R + W > N guarantees up-to-date reads, and R + W ≤ N allows stale reads. A write is acknowledged by W of the N replicas and a read contacts R replicas. If R + W > N, the two sets cannot be disjoint: by the pigeonhole principle they share at least R + W − N ≥ 1 replica. That replica holds the latest acknowledged write, and because the reader keeps the version with the highest timestamp, it returns that write. If R + W ≤ N, the reader can pick R replicas that are all outside the write set, and then it sees only older versions. Table 3 shows both cases: with W = 2 and R = 1 the read of {C} returned the old value, while with W = 2 and R = 2 every pair of nodes included A or B and returned the new one. Overlap alone gives the latest acknowledged value; full linearizability additionally needs a single order of writes and reads that do not run concurrently with an unfinished write (or read repair). This store gets the single order from the leaseholder, which handles every CP write and every linearizable read.

2. The trade-off on the minority partition. Under QUORUM_CP the system chose consistency over availability. Node C could not reach the leaseholder, so the write was

rejected with ReplicaUnavailableError (CAP-03) and no node stored it. Clients in DC-2 could not write, but no divergent history arose, and linearizable reads on the majority side stayed correct. Under ALWAYS_AP the system chose availability. C accepted the write with acks_received = 1 (CAP-04), so DC-1 and DC-2 held different values during the partition. After the partition healed, anti-entropy resolved the conflict with Last-Write-Wins (CAP-05, CAP-06): the version with the later timestamp survived and the other write was silently lost. A union merge, keeping both versions as siblings for the application, would avoid that loss at the cost of more complex reads.

Known Limitations

1. A QUORUM_CP write that fails can stay on the followers that did acknowledge it; there is no commit index or rollback. A replicated log (Raft, as in CockroachDB) would prevent this.
2. The lease is static: node A is always the leaseholder, and nothing moves the lease when A fails. A real system would elect a new leaseholder after the old lease expires.
3. Last-Write-Wins depends on clocks. All nodes share one machine clock here, but across machines clock skew can let an older write win.
4. Anti-entropy sends the whole state every 200 ms. This is fine for a lab-sized store; real systems exchange Merkle-tree digests.
5. State is kept in memory only, and the benchmark runs over localhost, so it shows no real network delay.

Appendix A

Package Versions (pip freeze)

```
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0
```

Appendix B

How to Reproduce

```
python3 -m pytest tests/test_cap.py -v        # 18 CAP tests
python3 -m pytest tests/ -q                   # all 35 tests
python3 perf_cap_bench.py                     # PACELC table -> bench_results/cap_bench.json
```